

Codex for Claude Code Users: A Practical Guide to Running Two Coding Agents in One Repository

September 2026

Table of Contents

1	Introduction	3
1.1	What this primer covers	4
1.2	What this primer is not	4
1.3	What you should already know	5
2	Two harnesses, one repository	5
2.1	The model is a decision component, not the whole agent	6
2.2	The harness supplies the working loop	6
2.3	The interface is where you steer the session	7
2.4	Tools and the execution environment define the agent’s reach	8
2.5	Coexistence is a repository practice, not a parity claim	9
2.6	A compact diagnostic model	9
3	First-session setup	10
3.1	Install and authenticate without changing the repository	10
3.2	Inspect the repository before asking for edits	11
3.3	Choose a deliberately small first task	12
3.4	Use a review loop, not a hand-off	12
3.5	End the first session with a reproducible hand-off	13
4	Instruction systems	14
4.1	Separate the repository contract from the harness adapter	14
4.2	What <code>CLAUDE.md</code> and <code>AGENTS.md</code> mean in this setup	15
4.3	The thin bridge pattern	15
4.4	Shared guidance without a second manual	16
4.5	Codex configuration is a supplement, not the project contract	16
4.6	A practical instruction audit	17
5	Workflow differences	17
5.1	Exploration: establish the map before choosing a route	18
5.2	Planning: make the change boundary explicit	19
5.3	Implementation: constrain scope, preserve local idioms	19
5.4	Testing: treat verification as part of the task	20

5.5	Review: review the diff, not the confidence	21
5.6	Continuation: preserve state deliberately	22
5.7	Git: use the working tree as the contract	22
5.8	Parallel work: divide by contract, not by file count	23
5.9	A practical cross-harness loop	23
6	Models and reasoning effort	24
6.1	What the controls mean	24
6.2	A dated snapshot of the current Codex models	25
6.3	Choose by task shape	26
6.4	Exploration and planning usually need less than implementation of ambiguity	27
6.5	A model-selection workflow	28
6.6	Fast, strong, and specialized are not permanent categories	29
6.7	Cost and latency are part of engineering quality	29
6.8	A compact selection rule	30
6.9	Verification notes for the final edition	30
7	Feature translation	30
7.1	Start with the job, not the feature name	31
7.2	Skills: reusable workflow knowledge	33
7.3	Hooks: automation around the agent	34
7.4	MCP: same protocol, different connection	35
7.5	Subagents: delegation is not parallelism by itself	36
7.6	Permissions and sandboxing are different controls	37
7.7	A migration rule of thumb	38
8	Prompting patterns	38
8.1	Exploration before editing	39
8.2	Bounded implementation	39
8.3	Review an existing diff	40
8.4	Ask for evidence, not a confident narrative	40
8.5	Prompts for delegated work	41
8.6	Recovery patterns	41
8.6.1	The agent changed the wrong files	41
8.6.2	The tests fail after the change	42
8.6.3	The agent claims a tool or capability is unavailable	42
8.6.4	The context is stale or contradictory	42
8.6.5	The agent is over-scoping the task	43
8.6.6	The agent is too hesitant	43
8.7	A compact prompt checklist	43
9	Migration cookbook	44
9.1	Add the smallest useful <code>AGENTS.md</code>	44
9.2	Choose a shared-instruction strategy	45
9.3	Roll out in a branch, one capability at a time	46
9.4	Port integrations by behavior, not by filename	47

9.5	Preserve a clean rollback path	48
10	Safety, cost, and operational discipline	48
10.1	Separate approval from sandboxing	48
10.2	Protect secrets and external systems	49
10.3	Control cost with task boundaries	49
10.4	Make generated changes observable	50
10.5	Establish an operating rhythm	50
10.5.1	Safety and operational checklist	50
11	What to measure	51
11.1	Define the comparison before running it	51
11.2	Use a task matrix that resembles real work	52
11.3	Score correctness before speed	52
11.4	Record the review burden	53
11.5	Report distributions, not a single victory lap	54
11.6	A practical evaluation record	54
11.6.1	Fair bake-off checklist	55
12	Conclusion and quick reference	55
12.1	Quick-reference: adding Codex to an existing repository	55
12.1.1	Minimal migration	55
12.1.2	Task-start prompt	56
12.1.3	Preflight checklist	56
12.1.4	Postflight checklist	56
12.1.5	Choosing a harness or model setting	56
12.1.6	Volatile-claim verification list	57
13	References	57

This is a long-form, operational guide for Claude Code users who want Codex as a second coding-agent harness while keeping their existing repository workflow intact.

Product snapshot · September 10, 2026

1 Introduction

If you already use Claude Code, the first few minutes with Codex can be oddly confusing. The terminal looks familiar. Both tools can inspect a repository, edit files, run commands, and explain a diff. You can give both of them a task such as “add a test for this bug” and get something that looks like the same workflow.

Then the similarities stop being useful. The instruction file may have a different name. A command that Claude Code would run immediately may produce an approval request in Codex, or be unavailable in the current execution environment. A model name may describe

a different thing from the product or interface you were comparing. A prompt that worked well in one harness may leave the other with too little context about scope, verification, or when it is allowed to edit.

This primer fills that gap. It is a translator’s guide for experienced Claude Code users who want Codex as a second coding-agent harness without turning an existing repository into a compatibility project. The goal is not to declare a winner. The useful question is: given this repository, this task, and these constraints, how do you make either harness behave predictably?

1.1 What this primer covers

The guide builds a working mental model before it gives you recipes. It explains the layers that people often collapse into the word “Codex” or “Claude,” then maps the practical differences that matter when you move between them:

- how a model, an agent harness, a user interface, and tools fit together;
- how Claude Code and Codex can share a repository while keeping their own instruction conventions;
- how to choose a harness and model for a task rather than treating the product name as the decision;
- how to translate familiar concepts such as `CLAUDE.md`, skills, hooks, Model Context Protocol (MCP), subagents, approvals, and sandboxing;
- how to write prompts that state scope, authority, and verification clearly;
- how to inspect, test, and review changes when two agents may work on the same codebase; and
- how to evaluate the two workflows using real tasks instead of anecdotes.

The examples use ordinary repository work: investigating a failing test, changing a build recipe, updating documentation, and reviewing a multi-file refactor. You do not need to adopt every suggested convention. The point is to make the important choices visible.

1.2 What this primer is not

This is not a complete reference manual for either product. Commands, configuration keys, available models, plan entitlements, usage limits, and interface behavior can change. Where an exact product detail matters, treat it as a dated claim and check the current official documentation before relying on it in automation or team policy.

It is also not a feature-by-feature promise that every Claude Code capability has a Codex

equivalent, or vice versa. Similar labels do not guarantee similar semantics. A “skill,” “hook,” or “subagent” may be implemented differently, may require a different configuration, or may have no direct counterpart. The safe approach is to compare the behavior you need, such as “run this check before committing” or “delegate this independent investigation,” and then find the mechanism that provides that behavior in the chosen harness.

Finally, this guide does not ask you to rewrite a working Claude Code setup. Claude Code can remain the primary harness. A thin `AGENTS.md` can give Codex a repository entry point, while shared build commands, tests, and neutral project documentation remain shared assets. You should only split instructions or workflows when the difference is real and worth maintaining.

1.3 What you should already know

You should be comfortable working in a terminal, reading a git diff, running a project’s tests, and making ordinary changes across several files. You should know what your repository’s `CLAUDE.md` says and understand the purpose of the tools it mentions. Familiarity with skills, hooks, MCP, and agentic coding will make the translations shorter, but the guide introduces each term when it becomes relevant.

You do not need to know Codex’s configuration syntax in advance. You do need to bring the habits that make any coding agent safe to use: give it a bounded task, keep the working tree understandable, inspect its proposed changes, and run the checks that establish whether the result is correct.

2 Two harnesses, one repository

The most useful starting point is to stop comparing “Claude” and “Codex” as if each were a single interchangeable object. In a coding session, at least four layers are involved:

1. the **model**, which generates text and tool-use decisions;
2. the **agent harness**, which maintains the task loop and turns those decisions into work;
3. the **interface**, through which you start the session and observe or steer it; and
4. the **tools and execution environment**, which determine what the agent can inspect or change.

These layers interact, but they are not synonyms. Two sessions can use the same underlying model and still behave differently because the harness supplies different instructions, exposes different tools, or applies different approval rules. Conversely, the same harness can behave differently after you change the model, reasoning setting, working directory, or sandbox.

2.1 The model is a decision component, not the whole agent

The model is the language-and-reasoning component that predicts the next response or action from the context it receives. In a coding task, that context can include your prompt, repository instructions, files, command output, previous tool results, and the harness’s own rules. The model can propose “inspect the test,” “edit this function,” or “run the test suite,” but it does not by itself define how those actions happen.

For example, suppose you ask:

The parser accepts a trailing comma in arrays but rejects one in objects.
Find the cause and add a regression test.

The model may infer a sensible investigation: locate the parser, compare the two grammar paths, reproduce the failure, and inspect related tests. The harness decides how to expose the filesystem and shell, whether the model may edit immediately, how to present command output, and when to ask for approval. If the session has no way to run the test suite, a capable model cannot manufacture that missing capability.

Model choice still matters. Different models can vary in coding accuracy, speed, context handling, and ability to sustain a multi-step investigation. Reasoning controls can also change how much effort a model spends before acting. Those are important choices, but they are only one part of the observed workflow. A strong model in a poorly scoped session can make a larger mess than a less powerful model given clear boundaries and a reliable verification loop.

Exact model names, availability, limits, and pricing are volatile product facts. Do not bake an undated model table into repository instructions. Keep task-level guidance such as “use a faster model for a small formatting change; use a deeper reasoning setting for an unfamiliar concurrency bug” separate from the current product catalogue. The model-selection chapter will return to this distinction.

2.2 The harness supplies the working loop

The agent harness is the program that turns a conversation into an agentic coding session. It loads instructions, assembles context, presents tools, interprets the model’s requested actions, applies permission checks, records results, and keeps the loop moving until the task is complete or it needs your input.

Claude Code and Codex are different harnesses. They overlap substantially: both can be used for repository exploration, implementation, testing, and review. That overlap lets you move familiar tasks between them. It does not imply identical defaults or feature semantics.

Imagine that both harnesses receive this request:

Inspect the build failure, explain the root cause, and propose a fix. Do not edit files yet.

One harness may naturally spend the session in an investigation-and-plan phase. Another may inspect the failure and then ask whether it should apply the proposed patch. Both outcomes can be reasonable. Your prompt should make the important boundary explicit, and you should verify that the harness honored it before proceeding.

The harness also determines how repository guidance is discovered. Claude Code users commonly put project-specific instructions in `CLAUDE.md`. Codex uses its own instruction-file conventions, including `AGENTS.md` as the natural compatibility entry point for a repository. The names are not interchangeable by magic: if Codex never loads `CLAUDE.md`, putting all of your operational rules there does not create a reliable Codex workflow.

That is why a thin bridge is usually better than a second, complete instruction manual. A repository can keep `CLAUDE.md` as the canonical project document and add a small `AGENTS.md` that points Codex to it and states any Codex-specific discovery or safety notes. If the two files start repeating the same long policy, they will drift. If they contain only a pointer and a few genuinely harness-specific instructions, the maintenance burden stays small.

For example, a bridge might look like this:

```
# Repository instructions
```

```
The canonical project instructions are in [ `CLAUDE.md` ] (CLAUDE.md). Read and follow that file for repository work.
```

```
This file is a thin compatibility layer for agents that discover `AGENTS.md`. Keep shared build, test, and style guidance in the canonical instructions.
```

This does not claim that Codex and Claude Code will interpret every instruction identically. It gives both sessions a clear route to the repository's shared intent, while leaving room to state a real difference when one exists. For instance, an instruction about a particular approval mode belongs with the harness that implements that mode; a command for building the project belongs in shared project guidance.

2.3 The interface is where you steer the session

The interface is the surface you use to start, observe, and control the harness. It might be a command-line session, an editor integration, or another product surface. The interface affects discoverability and ergonomics: how you select a model, ask for a plan, see a diff, resume a session, or respond to an approval request.

Do not infer execution capability from interface appearance. A command-line interface may still be operating inside a restricted sandbox. An editor panel may show a plan without granting permission to write. A session started with one product surface may have different

authentication, network access, or model availability from a session started with another. When a task behaves unexpectedly, identify which interface launched the harness before blaming the model.

As a concrete example, “run the integration tests” can fail for several unrelated reasons:

- the model chose the wrong command;
- the harness could not find the repository’s test instructions;
- the interface started the session in the wrong working directory;
- the execution environment lacks a service, dependency, or network route; or
- an approval or sandbox rule prevented the command from running.

The final error may look similar in the conversation, but the fixes are different. Ask which layer failed.

2.4 Tools and the execution environment define the agent’s reach

Tools are the capabilities the harness exposes to the model: reading files, searching text, applying patches, running shell commands, inspecting git state, calling an external service, or interacting with a browser. The execution environment is where those tools run and what they can reach. It includes the working directory, installed dependencies, containers, environment variables, network policy, credentials, and filesystem restrictions.

This distinction matters because “the agent can use shell” is not a complete statement. Shell access might mean read-only commands, commands that run only after approval, or commands inside a sandbox with no network and a limited filesystem. A tool may be present but unable to reach a service. A repository instruction may describe a container command that is valid on one machine and unavailable on another.

Keep two questions separate:

- **Approval:** may this proposed action proceed? An approval request is a control decision about a particular action.
- **Sandbox:** what can the execution environment permit even if the action is approved? A sandbox can still block network access, writes outside the workspace, or access to a required service.

For example, if Codex asks for approval to run a package-install command, approving it does not necessarily mean the command can write to every system directory or reach the package registry. Conversely, a permissive sandbox does not mean the harness will run every command without asking. Confusing these concepts leads to bad debugging and unsafe assumptions.

2.5 Coexistence is a repository practice, not a parity claim

Claude Code and Codex can work on the same repository because the repository already contains the most important shared assets: source code, tests, build commands, version history, and project conventions. They do not need identical internal workflows to share those assets.

The safe coexistence pattern is deliberately modest:

1. Keep project facts and shared conventions in a canonical location.
2. Give each harness a clear instruction-file entry point.
3. Put only real harness-specific behavior in harness-specific guidance.
4. Ask the agent to inspect the working tree before making a broad change.
5. Review the diff and run project checks after every meaningful task.

Suppose Claude Code has just added a database migration and left the working tree with a migration file, a schema change, and tests. You open Codex and ask it to “finish the migration.” That phrase is underspecified. Codex may create a second migration, alter the existing one, or focus on tests while missing the deployment note. A safer handoff names the current state and the allowed scope:

`Review the current working tree after another agent's database migration work.
Do not rewrite the existing migration. Identify missing tests or documentation,
make only the smallest changes needed to complete the task, and run the
migration-specific checks. Report any uncertainty before changing schema files.`

The repository is shared; the session state is not necessarily shared. Do not assume that Codex knows what Claude Code discussed earlier, that it has the same context window, or that it will interpret an informal handoff the same way. The durable handoff is the working tree, the git diff, the tests, and a short written note that states what remains.

Coexistence also means accepting asymmetry. One harness may be more convenient for a particular tool integration, permission policy, or long-running workflow. One model may be better suited to a particular investigation. One environment may have the credentials or services the task requires. Use the mechanism that fits the task, and make the result legible to the other harness through files, commands, tests, and explicit notes.

2.6 A compact diagnostic model

When a session goes wrong, classify the failure before changing prompts or models:

<code>model</code>	<code>-> reasoning, code generation, action proposals</code>
<code>harness</code>	<code>-> context assembly, tool loop, instructions, approvals</code>
<code>interface</code>	<code>-> session controls, visibility, selection, continuation</code>
<code>tools/env</code>	<code>-> files, commands, services, credentials, execution limits</code>
<code>repository</code>	<code>-> source, tests, build rules, durable handoff state</code>

If the generated patch is logically wrong, inspect the model’s reasoning and the context it received. If the agent ignored a repository rule, inspect instruction discovery and harness behavior. If it proposed the right command but could not run it, inspect tools, approval, and sandbox constraints. If the second agent misunderstood the first agent’s work, inspect the diff and handoff rather than assuming a model upgrade will solve the coordination problem.

This layered model is the foundation for the rest of the primer. The next sections turn it into an initial setup, an instruction-file strategy, and task workflows that let you switch harnesses without losing control of the repository.

3 First-session setup

Your first Codex session should not begin with a large feature request. The useful first question is whether Codex can understand the repository, respect its boundaries, and leave you with a diff you can explain. Treat the first session as a controlled integration test for a second harness.

The exact installation, authentication, model, and configuration commands are product details that change. The examples in this section use the command name `codex` where an executable is needed; check those commands against the current official Codex documentation before you publish or automate them. This guide intentionally avoids turning a volatile login flow into repository policy.

3.1 Install and authenticate without changing the repository

Install Codex using the current supported path for the interface you intend to use: command-line, editor integration, or another supported client. Keep the installation and authentication step separate from repository changes. You want to be able to tell whether a later problem comes from the client, the account, the repository instructions, or the task itself.

The following commands are Codex CLI examples, not a universal installation recipe for every Codex surface. After installation, use the client’s built-in help or diagnostics before opening a real task. This confirms which commands and flags exist in the version you actually installed.

```
# The executable name and available flags can change between Codex releases;  
# consult the current official documentation if this command differs.  
codex --help
```

The CLI, IDE extension, ChatGPT desktop app, ChatGPT web experience, and Codex cloud are related surfaces with different controls. The CLI exposes commands such as `codex`, `codex exec`, `/model`, `/permissions`, and `/review`; an IDE or desktop surface exposes equivalent

choices through its own controls, while cloud work runs in a hosted environment. Use the [Codex CLI guide](#) for terminal commands and the documentation for the specific surface when the execution environment matters.

Authentication may be browser-based, environment-variable-based, or managed by the host application. Do not put credentials in `CLAUDE.md`, `AGENTS.md`, `.codex/config.toml`, prompts, or committed shell scripts. If the repository needs a service token for tests, use the project’s existing secret-management procedure and confirm that the token is not included in command output or the diff.

Before the first write-capable session, establish what the client is allowed to do. “Approval” means whether you permit a proposed action. “Sandbox” means what the execution environment permits even if the action is approved. They are related, but they are not the same control. A command can require approval, and an approved command can still fail because the sandbox blocks network access, a filesystem path, a container runtime, or another capability.

The names and defaults for these controls vary by Codex interface and release. Treat any command-line flags, configuration keys, or UI labels as provisional until checked against the version in use. Start with the narrowest permissions that let the task run. Widen them only when a specific, understood operation needs it.

3.2 Inspect the repository before asking for edits

Open Codex from the repository root, or explicitly tell it which directory is the project root. The root matters because instruction files, build commands, relative paths, and git state are all interpreted in relation to the project.

Do a human inspection first. You need to know whether the worktree is clean, which files are already modified, and where the project describes its normal build and test commands.

A useful baseline is:

```
git status --short
git branch --show-current
rg --files \
  -g 'CLAUDE.md' -g 'AGENTS.md' -g 'README*' -g 'Justfile' \
  -g 'Makefile' -g 'package.json' -g 'pyproject.toml' | sort
```

The `rg` command is a discovery aid, not a universal project contract. Use the build system the repository already documents. Do not create a new package manager, test runner, or formatting convention merely because Codex knows one.

Your first prompt should request inspection only. Ask for the repository layout, applicable instructions, likely validation commands, and any concerns about the current state. Make

the no-edit boundary explicit:

You are in the repository root. Do a read-only orientation pass.

Inspect the applicable instruction files, the top-level README, the relevant build/test configuration, and the current git status. Do not edit files, run destructive commands, install dependencies, commit, or contact external services.

Report:

1. Which instruction files you found and which appear to apply.
2. The repository's main build, test, lint, and formatting commands.
3. The files and directories relevant to the next small documentation task.
4. Any dirty-worktree changes you will leave untouched.
5. Assumptions or missing information that should be resolved before editing.

This prompt tests more than file listing. It tests whether the agent can identify instructions as instructions, distinguish existing changes from its own work, and state uncertainty instead of silently filling gaps. If the answer invents a test command, claims that it changed nothing without showing how it checked, or misses an obvious instruction file, stop and fix the setup before giving it write access.

3.3 Choose a deliberately small first task

The first write should be reversible and easy to review. Updating one sentence in a document, adding a focused test, or correcting a narrow configuration comment is usually better than beginning with a cross-cutting refactor. The task should exercise the repository's normal workflow without making the first session depend on every tool integration at once.

Give Codex a bounded objective, explicit exclusions, and a validation request. For example: Make one narrowly scoped documentation change: clarify the sentence that describes how the project builds its HTML output.

Constraints:

- Read the applicable instruction files before editing.
- Edit only the relevant Markdown source and nothing else.
- Preserve the existing voice and heading structure.
- Do not update generated HTML/PDF files in this task.
- Do not change build configuration, dependencies, or unrelated wording.

After editing, show the diff, run the cheapest relevant validation command if one is documented, and report anything you could not verify. Do not commit.

The exclusions are important. A coding agent often sees generated files, formatting opportunities, stale documentation, or a failing unrelated test and tries to make the repository "better" while it is there. That may be useful on a later task, but it makes the first diff harder to attribute. Scope discipline is part of what you are testing.

3.4 Use a review loop, not a hand-off

The basic loop is:

`inspect -> propose -> edit -> validate -> review -> accept or revise`

Do not treat a successful tool call as evidence that the result is correct. Review the diff yourself. At minimum, check the changed paths, the actual content, whitespace, generated files, and the validation output.

```
git status --short
git diff --check
git diff -- path/to/the/expected/file.md
```

Then run the repository's documented validation command. For a small Markdown change that might be a link checker or a render preview; for code it might be the focused test followed by the normal test suite. A command that is not documented, that installs software, or that reaches a network service should be treated as a separate decision rather than an automatic part of the loop.

Ask Codex to explain the result in terms of evidence, not confidence:
Review the current diff against the original request.

Report:

- every file changed;
- which requested constraints you verified and how;
- commands run and their exit status;
- checks you could not run and why;
- any behavior or wording that remains uncertain.

If the diff exceeds the requested scope, propose a correction before making more edits. Do not commit.

If the result is wrong, give a targeted correction and keep the same review loop. Do not respond to an incorrect first attempt by broadening the task. A small, explicit correction tells you whether the agent can recover without losing the original constraints.

3.5 End the first session with a reproducible hand-off

Before you close the session, ask for a short hand-off containing the final state, validation performed, unresolved questions, and the next safe action. You should be able to resume the work from the repository and the diff rather than relying on a vague memory of the conversation.

Summarize this session for another engineer:

- the original objective;
- files changed and why;
- validation commands and results;
- assumptions and unresolved issues;
- any follow-up work that is explicitly out of scope.

Do not modify files or commit while writing the summary.

A first session has gone well when you know what Codex did, what it did not do, which instructions it followed, and which parts still need human judgment. It has not gone well

merely because the client started, a model responded, or the task ended without an error message.

4 Instruction systems

Claude Code and Codex both use project instructions, but an instruction file is not a portable command language. `CLAUDE.md` and `AGENTS.md` are conventions used by different harnesses; they do not automatically have identical discovery rules, precedence, inheritance, or feature support. The practical problem is to give two agents one repository contract without maintaining two conflicting manuals.

4.1 Separate the repository contract from the harness adapter

Start by classifying an instruction:

Kind of guidance	Examples	Best home
Repository-wide	build commands, test requirements, source/output rules, coding style, generated-file policy	Shared project guidance or the project’s canonical instruction file
Claude Code-specific	Claude-only skills, hooks, commands, or interaction conventions	<code>CLAUDE.md</code> or Claude-specific configuration
Codex-specific	Codex approval, sandbox, model, or session controls	<code>AGENTS.md</code> or Codex configuration
Task-specific	Constraints for one migration, experiment, or directory	A local instruction file or the task prompt

The most important distinction is between a policy and an adapter. “Run the focused test before declaring success” is repository policy. “Use this harness’s skill command to discover a workflow” is an adapter detail. Keep the policy stable and make each harness adapter as thin as possible.

Do not copy a large `CLAUDE.md` into `AGENTS.md` just to make both agents see the same words. Duplication creates two sources of truth. The files will drift, and the drift will be hardest to notice when the instructions concern safety, generated outputs, or validation.

4.2 What `CLAUDE.md` and `AGENTS.md` mean in this setup

For this guide, use these terms precisely:

- `CLAUDE.md` is the project instruction file associated with Claude Code in the existing repository.
- `AGENTS.md` is the Codex-facing compatibility or instruction file.
- “Instruction file” is the neutral term when the advice applies to either harness.
- A referenced file is not necessarily an included file. A pointer only works if the harness reads the target and applies its contents.

The current [Codex instruction-discovery guidance](#) (checked 10 September 2026) says that Codex checks `AGENTS.override.md`, then `AGENTS.md`, then configured fallback names in each directory from the project root to the current directory. It includes at most one file per directory and concatenates the discovered files from root to leaf. The default combined instruction limit is 32 KiB. These are useful facts for designing a repository, but they remain product behavior: recheck them when a long-lived workflow depends on them. Project-wide guidance belongs at the repository root, and deeper directory-specific instructions should be explicit.

The same caution applies to configuration keys. A key accepted by one Codex release may be renamed, moved, or removed in another. Configuration should support the instruction design; it should not be the only place where a safety or validation rule exists.

4.3 The thin bridge pattern

If Claude Code remains the primary harness, the least disruptive adaptation is a thin root-level `AGENTS.md` that names the existing `CLAUDE.md` as canonical. The bridge should explain the arrangement, not repeat the project manual. This repository uses that pattern:

```
# Repository instructions
```

```
The canonical project instructions are in [ `CLAUDE.md` ] (CLAUDE.md). Read and follow that file for repository work.
```

```
This file is intentionally a thin compatibility layer for agents that discover `AGENTS.md`, so the project can support Codex without maintaining a second copy of its instructions.
```

The pattern has three advantages. Claude Code keeps its existing source of truth. Codex has a conventional entry point to discover. Reviewers can see the cross-harness adaptation in a small, obvious diff.

It also has a failure mode: the bridge assumes that Codex follows the reference to `CLAUDE.md`. Test that assumption in a read-only first session. If the agent does not read the referenced file,

move only the genuinely shared rules into a neutral document or reproduce a small, carefully maintained subset in `AGENTS.md`. Do not assume that a Markdown link is an instruction include.

The bridge is not a claim that Claude Code and Codex have feature parity. It only establishes where a repository-level reader should begin. Harness-specific skills, hooks, permissions, and model controls still need their own setup.

4.4 Shared guidance without a second manual

There are three reasonable designs. Pick one deliberately instead of allowing the repository to accumulate all three.

The first is the thin bridge used here: keep the existing canonical file and make the other harness point to it. This minimizes change and is a good default when the primary harness already has a mature instruction set. Verify that the secondary harness actually follows the reference, and keep the bridge itself free of rules that exist nowhere else.

The second is a neutral shared document with small adapters. For example, a repository could keep build and validation policy in `PROJECT_GUIDE.md`, then have each harness instruction file tell its agent to read that guide. This is clean when both harnesses are first-class and the shared contract is large. It adds another file to discover, so the first-session inspection must verify that both agents read it.

The third is a deliberately duplicated minimal contract. Each harness file contains the same short list of non-negotiable rules, while the detailed tool-specific guidance stays separate. This can be appropriate when a harness cannot reliably follow references, but it creates a maintenance obligation. Keep the duplicated section short and make drift detectable in review.

For this project, the first design is sufficient. The repository already has a substantial `CLAUDE.md`, and the desired Codex adaptation is intentionally lightweight. A migration should not reorganize every instruction merely to introduce a second reader.

4.5 Codex configuration is a supplement, not the project contract

Codex may have user-level and repository-level configuration for model choice, approval behavior, sandboxing, or instruction-file fallback. Those settings are more volatile than repository prose and may differ by client. Keep them small, documented, and safe to ignore when a user runs another interface.

At the time of drafting, this repository contains the following fallback configuration:

```
# Additional filename tried only when AGENTS.md is missing.  
project_doc_fallback_filenames = ["CLAUDE.md"]
```

The intent is narrow: if the normal Codex-facing file is absent, the existing Claude Code instruction file remains discoverable as a fallback. It does not turn `CLAUDE.md` into a universal Codex configuration format, and it does not add `CLAUDE.md` as a second project instruction source when `AGENTS.md` is present. The current configuration reference also says that project-local `.codex/` layers are skipped for untrusted projects, so trust status is part of the setup rather than an implementation detail.

Do not put model preferences, approval choices, or secrets into shared project instructions unless every contributor should inherit them. A developer who needs a more permissive sandbox for a local experiment should be able to make that choice locally without changing the repository's review policy for everyone else.

4.6 A practical instruction audit

When adding Codex to an existing Claude Code repository, audit the instruction system in this order:

1. Identify the current canonical project instructions and read them before writing an adapter.
2. Separate repository policy from Claude-specific features.
3. Add the smallest Codex-facing entry point that can be tested.
4. Check for contradictions between `CLAUDE.md`, `AGENTS.md`, local files, and user-level configuration.
5. Ask each harness, in a read-only session, which instructions it found and which rules it believes apply.
6. Run one bounded write task and inspect whether the agent followed the shared build, scope, and validation rules.

The audit is complete when both agents can answer the same repository questions and still retain their own harness-specific workflows. You do not need identical prompts or identical feature sets. You do need one understandable source of truth for what the repository considers a correct change.

5 Workflow differences

If you already use Claude Code, the first Codex session can feel familiar enough to be misleading. You still describe a goal, let an agent inspect a repository, approve or reject actions,

review a diff, and run tests. The important differences are in the defaults and in the places where the harness decides how to gather context, when to ask for permission, how much reasoning to spend, and how to continue after the first pass.

Treat Codex as a second harness rather than as a different spelling of the same workflow. The transferable skill is the engineering loop: establish the task boundary, inspect the relevant system, make a small change, verify it, and review the result. The parts you need to recalibrate are the hand-off between those stages and the amount of context you make explicit.

5.1 Exploration: establish the map before choosing a route

Claude Code users often begin with a natural-language request and allow the agent to discover the repository while working. That can work well when the repository instructions, naming conventions, and test layout are already familiar. With a second harness, exploration has another purpose: it tests whether the new harness has found the same sources of truth that you rely on in Claude Code.

Start a Codex task with a read-only reconnaissance pass when the repository is unfamiliar or the change crosses several components. Ask it to identify the relevant instruction files, entry points, tests, build commands, generated files, and local conventions. Ask for paths and evidence, not just a summary. For example:

`Inspect this repository without modifying files. Identify:`

1. The project instruction files and which ones apply to this directory.
2. The likely entry points for the authentication flow.
3. The tests that exercise the flow.
4. The commands used for formatting, testing, and building.
5. Any generated or vendored files that should not be edited directly.

`Give me a short map with file paths and explain what you would inspect next.`

This prompt is useful when switching harnesses because it exposes instruction-discovery drift early. If Codex names a different build command, misses a nested instruction file, or treats a generated file as a source file, fix the context before asking it to implement anything. The same repository can produce different outcomes when the agents receive different local rules.

Exploration should be proportional to the task. For a one-line change in a well-known file, asking for a repository census creates noise and consumes context. For a migration or bug with several possible causes, skipping reconnaissance makes the first edit a guess. A good rule is to explore until you can state the change surface: the files that probably change, the tests that should fail or pass, and the commands that can prove the result.

Do not ask the agent to read every document “for context.” Large instruction files, generated

outputs, fixtures, and historical notes can crowd out the code that matters. Point it towards the relevant paths and ask it to follow references only when they affect the task. This is especially important when Claude Code and Codex use different instruction-file conventions: a thin compatibility file is useful only if the agent actually follows the path to the canonical guidance.

5.2 Planning: make the change boundary explicit

Planning has two meanings in agentic coding. It can mean a lightweight statement of intended edits before work begins, or it can mean a detailed implementation design with dependencies, alternatives, and test strategy. Both are useful, but they serve different tasks.

For a small change, ask for a compact plan and proceed quickly. For a risky change, require the plan to name invariants and verification steps. A useful plan answers four questions:

- What behavior changes?
- What behavior must remain unchanged?
- Which files or interfaces are in scope?
- How will you know the implementation is correct?

Claude Code users sometimes rely on a conversational plan that evolves as the agent explores. Codex can work that way too, but you get a more reliable hand-off if you distinguish discovery from commitment. Ask it to investigate first, then approve a plan once the evidence is visible: `First inspect the relevant code and tests. Do not edit files yet. Then propose the smallest implementation plan. Include the expected files, the tests you will add or update, and any uncertainty that could change the plan. Wait for my approval before making changes.`

This is not bureaucracy. It prevents a common failure mode in which the agent turns an ambiguous request into a broad refactor before it has located the existing abstraction. It also makes it easier to compare Claude Code and Codex: give both harnesses the same task boundary and inspect where their proposed approaches diverge.

Plans should not become a substitute for verification. A polished plan can still rest on a false assumption about the code. Once implementation starts, let the agent revise the plan when tests or source inspection contradict it. The useful distinction is between a plan that records current intent and a rigid script that prevents learning.

5.3 Implementation: constrain scope, preserve local idioms

The implementation stage is where familiar-looking behavior hides the biggest practical difference. An agent can produce a plausible patch while using the wrong abstraction, editing

generated output, or changing more files than the request warrants. The harness’s ability to inspect and modify files matters, but so does the instruction you give it about scope.

State the boundary in terms the repository can enforce. “Fix the parser” is weaker than “change the parser under `src/config`, preserve the public error type, add regression coverage for malformed input, and do not change the serialization format.” Include explicit exclusions when they matter: no dependency upgrades, no unrelated formatting, no generated-file edits, no API renaming.

Ask the agent to implement in a coherent slice rather than narrate every keystroke. Excessive micro-management can make the task brittle: the agent spends its context repeating instructions instead of checking the code. The opposite failure is an unbounded request such as “clean this up,” which gives the agent no principled stopping point. A good implementation prompt states the contract, the allowed surface, and the acceptance checks.

When Codex is working in a repository that Claude Code has already shaped, tell it which local pattern to follow. Point to an existing analogous implementation instead of describing the style abstractly:

```
Implement this using the same boundary and error-handling pattern as
`src/importers/json_importer.py`. Keep the public API unchanged. Add tests
next to the existing importer tests. Do not edit generated files or update
dependencies. Stop after the focused tests pass and report the full test
command you ran.
```

The example does more than prescribe style. It gives the agent a nearby source of truth and a stopping condition. If the two harnesses have different defaults for formatting, file discovery, or command approval, those explicit constraints reduce accidental divergence.

5.4 Testing: treat verification as part of the task

A test command is not a magic stamp of correctness. It is evidence about a particular slice of behavior under a particular environment. Ask the agent to choose tests that correspond to the changed contract, then widen verification when the change crosses a boundary.

For a focused bug fix, the useful sequence is usually: reproduce or write the failing case, make the smallest change, run the focused test, run the relevant package or subsystem tests, and run broader checks if the risk justifies them. For a documentation or configuration change, validation may mean rendering, linting, schema checking, or inspecting the generated artifact rather than running a unit-test suite.

Tell Codex what “done” means before it edits. Otherwise an agent may stop after a syntactically valid patch or report that a test could not run without distinguishing an environmental failure from a product failure. Ask it to report the command, result, and any limitation:

Verify the change in layers:

1. Add or update a regression test for the reported behavior.
2. Run the focused test file.
3. Run the relevant package test target.
4. If a check cannot run, report the exact command, error, and whether the limitation is environmental or caused by the patch.

Do not claim the task is complete based only on static inspection.

Claude Code and Codex may expose different approval or sandbox behavior around commands, especially when a test needs network access, containers, or writes outside the immediate project directory. Keep the conceptual distinction clear: approval determines whether the agent may take an action; the sandbox determines what the execution environment allows. A command can be approved and still fail because the sandbox blocks it, or it can be permitted by the environment while still requiring a deliberate approval decision.

Testing is also a useful way to compare agents without relying on impressions. Give each harness the same repository state and acceptance criteria. Compare the tests each one chooses, the failures it notices, and whether its final explanation accurately describes what ran. “It felt more capable” is weak evidence; a reproducible verification record is stronger.

5.5 Review: review the diff, not the confidence

An agent’s final summary is a useful index, not a substitute for review. The summary can be concise and accurate while omitting a subtle behavior change, or it can sound confident when a test was skipped. Review the actual diff and the evidence behind the claims.

Ask Codex for a review pass after implementation, including a fresh look at scope and failure modes:

Review the current diff as if you did not write it. Check:

- whether every changed file is necessary;
- whether the implementation preserves the existing public behavior;
- whether the tests cover the failure mode rather than only the happy path;
- whether error handling, input validation, and compatibility were changed;
- whether generated files or unrelated formatting slipped in.

Report findings by severity, then list the verification commands and results.

Do not modify files during this review.

A second agent can be particularly useful as a reviewer because it approaches the diff without the original implementation narrative. That does not make it automatically independent: both agents can inherit the same misleading issue description, test gap, or repository instruction. If the change is high impact, review the behavior from outside the agent loop as well. Run the relevant tests yourself, inspect interfaces, and check the final diff against the original request.

Use a clean review context when the task is complex. Continuing in the same session preserves useful context, but it also preserves the assumptions that produced the patch. A fresh session or a second harness can expose a missed edge case precisely because it has to reconstruct the reasoning from the repository and diff.

5.6 Continuation: preserve state deliberately

“Continue” is not one operation. It can mean continue in the same conversation, resume after a command failure, pick up an unfinished plan, or hand work to a different harness. Each form has a different context budget and a different risk of carrying forward a bad assumption.

Within one session, continuation is efficient when the agent has already inspected the right files and the next step is a direct consequence of the previous one. State the new checkpoint explicitly: what is complete, what failed, and what must happen next. Do not assume that a previous summary proves the repository still has the same state; files may have changed, tests may have produced generated artifacts, or another agent may have edited the branch.

When handing off from Claude Code to Codex, or the reverse, write a small hand-off note in the prompt rather than relying on hidden conversational context:

```
The previous agent implemented the change in the current working tree.  
Please inspect the diff before doing anything else. The intended contract is  
[contract]. The focused test passed, but the broader suite was not run because  
[reason]. Your job is to review the implementation, identify missing coverage,  
and make only necessary corrections.
```

This makes the working tree the primary state and the prompt a map to it. It also prevents a dangerous hand-off pattern: telling the second agent that the first agent “finished” when the only evidence is a conversational claim.

Context compaction or summarization introduces another reason to use checkpoints. Keep durable facts in files that belong in the repository, such as plans, issue notes, test output, or a hand-off document, only when the project benefits from that record. Otherwise include the essential facts in the next prompt. A compact checkpoint should name the current diff, the last successful command, the unresolved failure, and the next allowed action.

5.7 Git: use the working tree as the contract

Git is the most reliable shared language between Claude Code and Codex. Both agents can inspect a branch, diff, status, history, and tests, but you should not assume that either one will choose the same commit or worktree strategy by default.

Before implementation, establish the starting point with `git status` and inspect any existing changes. If the working tree is already dirty, tell the agent which changes belong to the

current task and which must remain untouched. “Do not overwrite my work” is useful, but naming the paths is safer. Ask for a diff review before staging or committing, especially when generated files are committed in the project.

A practical boundary looks like this:

Work only on the requested change. Preserve all pre-existing modifications.
Before editing, inspect ``git status`` and the existing diff. At the end, show the files changed and the relevant diff summary. Do not commit, reset, rebase, or discard changes unless I explicitly ask you to.

The exact commands and defaults for commits, branches, worktrees, and remote operations are product-specific and can change. Treat them as configuration and workflow choices, not as inherent properties of the model. Check the current Codex CLI documentation before relying on a particular workflow.

For parallel work, isolated worktrees or branches reduce accidental interference. They do not eliminate merge risk: two agents can make incompatible assumptions even when they edit different files. Keep parallel tasks partitioned by ownership and contract, then run an integration test and review the combined diff. If two agents must touch the same interface, serialize the design decision before parallelizing implementation.

5.8 Parallel work: divide by contract, not by file count

Parallel agents are useful when the work has separable outcomes. “One agent per file” is a poor partition if all files implement one coupled behavior. Better divisions are “research the existing API,” “add focused regression coverage,” “implement the backend adapter,” and “review the resulting diff,” provided each role has a clear output and a defined boundary.

Give each parallel worker the same repository rules and a narrow task contract. Tell workers whether they may edit files, whether they should commit, and how they should report conflicts. Keep a single owner for integration. The owner should inspect every branch or worktree, resolve interface differences, run the full relevant verification, and decide which changes belong in the final patch.

Parallelism increases throughput only when coordination costs stay below the time saved. It is a poor fit for exploratory debugging where each new observation changes the next step, or for a small task whose review cost exceeds its implementation cost. It also increases the chance of duplicated edits, inconsistent terminology, and tests that pass in isolation but fail together.

5.9 A practical cross-harness loop

For most repository work, the following loop transfers cleanly from Claude Code to Codex:

1. Establish the working-tree boundary and read the applicable instructions.
2. Explore until you can name the change surface and verification path.
3. Ask for a plan when the task has meaningful ambiguity or risk.
4. Implement the smallest coherent change using local patterns.
5. Run focused checks, then widen verification according to the change surface.
6. Review the diff and the evidence, ideally from a fresh context for risky work.
7. Record a checkpoint before handing the work to another session or harness.

The difference is not that one harness follows this loop and the other does not. The difference is how much of the loop each harness performs implicitly and how much you need to make explicit. If Codex is producing broad or surprising patches, tighten the task contract and require an inspection pass. If it is spending too much time describing obvious steps, shorten the plan and move to a focused test. Adjust the interaction pattern before concluding that the underlying model cannot handle the task.

6 Models and reasoning effort

Claude Code users often ask, “Which Codex model should I use?” That question is incomplete. The useful choice is a combination of model capability, reasoning effort, context available to the task, execution environment, latency, and cost. Exact model names and entitlements change; the task characteristics are more stable.

Think of model selection as choosing an engine for a job, and reasoning effort as deciding how long the engine should spend checking the route. A stronger model at a lower effort may be faster and more useful than a weaker model at maximum effort. A high-effort run can still produce a wrong answer if the repository context or acceptance criteria are wrong. Model controls improve the search; they do not replace a good task boundary and tests.

6.1 What the controls mean

The model is the underlying system generating decisions, tool calls, and text. Models differ in coding ability, instruction following, context handling, speed, availability, and cost. Those differences are not perfectly captured by a single intelligence ranking. A model that is excellent at a self-contained algorithm may be less useful for a large repository if it is slow to navigate or weak at maintaining constraints across many files.

Reasoning effort is a control over how much internal computation or deliberation the system allocates before producing an answer or taking the next action. The labels and exact behavior depend on the product surface. Conceptually, lower effort favors speed and throughput;

higher effort gives difficult tasks more room for decomposition, checking, and alternative consideration. It does not guarantee correctness, and increasing it can make a poor prompt more expensive without making it more precise.

In the current CLI model picker, the documented reasoning choices include Low, Medium, High, Extra High, Max, and Ultra. Other surfaces use different labels. Max gives one selected model more time for a difficult task; Ultra is a separate delegation mode that can use subagents for work that divides cleanly. Do not describe Max and Ultra as merely two points on one universal effort slider.

Context is the third control people often overlook. The model can only reason over the instructions, conversation, files, tool results, and other material available in the current context. More context is not automatically better: irrelevant logs and giant generated files can bury the contract and the code that matters. Curate context by pointing to the relevant paths, asking for targeted inspection, and preserving durable decisions in the repository when appropriate.

Execution capability is separate again. A model may be able to propose a correct command while the harness lacks permission, network access, credentials, or a suitable runtime to execute it. Keep model quality, reasoning effort, context quality, and execution permissions as separate variables when diagnosing a failure.

6.2 A dated snapshot of the current Codex models

The official [Codex model guidance](#) currently describes an Astra model alongside the GPT-5.6 family and previous-generation models. The following is a snapshot checked on 10 September 2026, not a promise that every model is available to every account, interface, or authentication method.

Model	Practical orientation	Good starting use
<code>gpt-6-astra</code> (Astra)	Highest-capability model for complex work across code, apps, and research	Hardest end-to-end tasks that need sustained reasoning and judgment
<code>gpt-5.6-sol</code> (Sol)	Highest-capability model in the current family	Ambiguous, high-value, multi-step coding, research, or security work
<code>gpt-5.6-terra</code> (Terra)	Balanced everyday model	Routine implementation, debugging, and repository work

Model	Practical orientation	Good starting use
<code>gpt-5.6-luna</code> (Luna)	Fast, lower-cost model in the family	Clear, repeatable transformations, extraction, and structured tasks
<code>gpt-5.3-codex-spark</code> (Codex Spark)	Text-only research preview focused on near-instant iteration	Very fast, narrowly scoped coding loops when the surface exposes it
<code>gpt-5.5</code>	Previous-generation frontier model	Existing configurations or tasks that specifically require it
<code>gpt-5.4</code> and <code>gpt-5.4-mini</code>	Previous-generation models approaching retirement in ChatGPT-authenticated Codex	Do not start new long-lived configuration around them without checking the retirement notice

The same documentation says that GPT-5.4 and GPT-5.4 Mini retire from Codex with ChatGPT sign-in on 31 August 2026, while API-authenticated use is not affected by that specific retirement. Treat that as a dated operational note: model names, availability, and migration advice belong in a maintained reference section, not in permanent repository instructions.

The practical selection rule is simpler than the catalogue. Start with Terra for ordinary work, move to Sol when ambiguity or the cost of a wrong decision justifies more capability, and choose Astra for the hardest end-to-end work across multiple tools or surfaces. Use Luna for clear high-volume tasks. Treat Spark as a specialized preview rather than the default for a long, open-ended task. Then choose reasoning effort independently.

6.3 Choose by task shape

Use the least expensive configuration that can reliably satisfy the acceptance criteria, then increase capability when evidence says the task needs it. The goal is not to use the strongest available model for every prompt. The goal is to match the configuration to the uncertainty and cost of being wrong.

Task shape	Starting configuration	Increase effort or capability when...
Mechanical edit with clear tests	Fast or lower-effort option	The change crosses an abstraction boundary or tests are weak
Familiar bug with a reproducible failure	General coding option at moderate effort	The cause is non-local, intermittent, or concurrent
Unfamiliar repository exploration	Stronger context handling at moderate effort	The architecture is ambiguous or the first map conflicts with the code
Multi-file refactor or API migration	Stronger model with deliberate planning	Compatibility constraints, generated code, or many callers are involved
Security-sensitive, data-sensitive, or high-impact change	Stronger model at higher effort plus independent review	The threat model is unclear or verification cannot cover the risk
Review of an existing patch	A configuration different from the implementer's when possible	The reviewer repeats the same assumptions or lacks enough context

This table is a starting heuristic, not a benchmark. A “simple” change in a mature repository can be harder than a large but well-specified migration. The cost of failure matters as much as the amount of code: a small permission change deserves more scrutiny than a large comment update.

6.4 Exploration and planning usually need less than implementation of ambiguity

High reasoning effort is most useful when the agent must resolve competing explanations, preserve many constraints, or make a design choice with expensive consequences. It is less useful when the task is already fully specified and the remaining work is mechanical.

For exploration, begin with enough capability to build a trustworthy map. Ask for evidence and paths. If the agent cannot distinguish the relevant subsystem from neighboring code, increase capability or narrow the search before asking for a larger answer. Spending maximum effort on an unbounded repository tour often produces a long but shallow inventory.

For planning, effort should track the number of plausible designs and the cost of choosing the

wrong one. A small bug with one obvious fix does not need a design essay. A compatibility migration, concurrency change, or schema transition does. Ask the agent to state assumptions and alternatives so that you can see whether extra reasoning is addressing real uncertainty.

For implementation, increase effort when the patch must preserve invariants across files, reason about edge cases, or modify code whose behavior is not fully captured by tests. Keep the task bounded. A powerful model with a broad mandate can still turn a focused fix into an unnecessary redesign.

For testing and review, capability matters because the agent must interpret failures and look for omissions rather than merely produce code. Independent review is often more valuable than repeatedly increasing the implementer’s effort. A second pass with a fresh context can challenge assumptions that a longer first pass has merely reinforced.

6.5 A model-selection workflow

Instead of memorizing a product matrix, use a short escalation loop:

1. Classify the task as mechanical, diagnostic, design-heavy, or high-impact.
2. Choose a fast or general configuration that can inspect the relevant code.
3. Set a clear acceptance test and ask for a bounded first pass.
4. Inspect the result and the evidence, not just the prose.
5. Increase reasoning effort or switch models only when the failure reveals a capability problem.
6. Re-run the same acceptance checks after changing the configuration.

This gives you a meaningful signal. If the first run failed because it never found the right file, a stronger model may not help until you improve the prompt or instruction layer. If it found the right code but repeatedly misses a cross-file invariant, more reasoning or a stronger model may be justified. If it produced the right patch but could not run tests, the problem is execution configuration rather than model choice.

You can make the escalation explicit in a prompt:
Start with a focused investigation and do not edit yet. If the cause is clear, propose the smallest fix. If there are multiple plausible causes, compare them using the existing tests and call sites. Do not switch to a broader redesign without explaining why the local fix would be unsafe.

The prompt helps the agent spend effort on uncertainty rather than on ceremony. You can then raise the effort setting for the same task if the investigation remains inconclusive, without changing the task contract.

6.6 Fast, strong, and specialized are not permanent categories

Product documentation may describe models using categories such as fast, general-purpose, reasoning-focused, coding-focused, or preview. Treat those categories as useful orientation, not permanent technical properties. A model can be renamed, retired, reclassified, made available in a different interface, or exposed with different controls. The same label may also behave differently across a chat product, command-line tool, application programming interface (API), or hosted execution environment.

For that reason, this primer keeps exact model names, prices, quotas, rate limits, context sizes, and plan entitlements in a dated snapshot rather than in its core workflow advice. Keep the durable guidance task-oriented, and recheck the official model and usage documentation before relying on any volatile detail.

Do not infer that a model available in one Codex surface is available in every other surface. Availability can depend on account, organization, region, product, authentication method, rollout stage, or API access. The final version should link to official documentation for these points and state the date checked.

6.7 Cost and latency are part of engineering quality

The cheapest run is not always the cheapest workflow. A low-cost configuration that creates three rounds of repair, review, and failed testing can cost more time and compute than one deliberate first pass. Conversely, using maximum effort for every search and formatting change wastes budget and slows feedback.

Estimate the cost of being wrong. For a reversible documentation edit, optimize for fast iteration. For a production migration, optimize for evidence and review. For an exploratory question, use a bounded investigation before committing to a long autonomous run. Track elapsed time, number of repair cycles, verification quality, and the final diff, not just the nominal model price.

When comparing Claude Code and Codex, keep the comparison fair. Use equivalent task descriptions, the same repository state, comparable permissions, and the same acceptance checks. Record whether a run needed human correction and whether the reported test results were accurate. Exact prices and quotas should be captured separately because they are volatile and can differ by product surface. Confirm which cost and usage metrics are exposed by the reader's chosen Codex interface before adding a comparison example.

6.8 A compact selection rule

Start fast when the task is bounded and mechanically verifiable. Move to a stronger model or higher reasoning effort when the task is ambiguous, cross-cutting, or expensive to get wrong. Improve the context and acceptance criteria before escalating when the failure is about repository discovery or execution. Add an independent review when the risk remains high after the implementation looks correct.

That rule keeps model selection subordinate to engineering discipline. The model is one component of the loop; the harness, instruction files, permissions, tests, and review process determine whether a plausible answer becomes a safe change.

6.9 Verification notes for the final edition

The concepts in these sections are intended to remain stable, but the final primer should verify and date-stamp any product-specific claims about:

- exact Codex model names, aliases, and model families;
- which models and reasoning controls are available in each interface;
- the meaning and range of reasoning-effort settings;
- context limits, rate limits, quotas, and usage accounting;
- prices, subscription terms, API billing, and plan entitlements;
- support for commits, branches, worktrees, remote operations, and parallel execution;
- current sandbox, approval, network, and container behavior.

Use official OpenAI documentation for those checks. Do not turn the verification list into a permanent matrix unless someone owns updating it.

7 Feature translation

If you already use Claude Code, the first Codex session can feel familiar enough to be misleading. Both tools can inspect a repository, edit files, run commands, use external tools, and work through a multi-step coding task. That does not make their controls interchangeable. A feature with the same informal name may have different discovery rules, different defaults, or a different boundary between the model and the harness.

The useful question is not “What is the Codex version of this Claude Code feature?” It is “What job was this feature doing, and which Codex mechanism, if any, does that job?” Sometimes the answer is a direct analogue. Sometimes the answer is a partial analogue that needs a different workflow. Sometimes the safe answer is that no equivalent exists and you

should use a repository, shell, or CI convention instead.

This section uses three labels:

- **Direct equivalent** means the same broad capability exists and can usually carry the same intent, although syntax and configuration may differ.
- **Partial equivalent** means the goal can be achieved, but the lifecycle, scope, or guarantees differ enough that a copy-and-paste migration is unsafe.
- **Non-equivalent** means there is no reliable one-to-one feature. Use a different layer of the system.

The labels describe the capability, not the product quality. A partial equivalent can be the better tool for a particular job. It simply requires a deliberate translation.

7.1 Start with the job, not the feature name

Claude Code users often carry a feature-shaped mental model into Codex. For example, they may ask, “Where do I put my hook?” before deciding what the hook was protecting. Was it formatting changed files? Blocking a dangerous command? Injecting context? Recording an audit event? Those are four different jobs and may belong in four different places.

A useful translation has four parts:

1. **Purpose:** what outcome did the Claude Code feature provide?
2. **Trigger:** what caused it to run: every prompt, a tool call, a lifecycle event, or an explicit request?
3. **Authority:** was it advisory, a gate that could block an action, or an action with its own side effects?
4. **Scope:** did it apply to one session, one repository, one user, or the whole organization?

For example, a Claude Code hook that runs a formatter after an edit has a different translation from a hook that rejects access to production credentials. The first can often move to a formatter command, pre-commit check, or CI job. The second needs a security boundary that does not depend on an instruction to the model.

The table below is a starting map. It is intentionally about responsibilities rather than exact command names or file paths.

Claude Code concept	Codex-facing	Status	Main caution
	translation		
Skills	Reusable Codex skill, plugin-provided workflow, or repository instruction	Partial to direct	Packaging, discovery, and invocation are surface- and release-dependent.
Hooks	Lifecycle automation, shell wrapper, git hook, formatter, or CI check	Partial to non-equivalent	Do not assume the same event model or blocking guarantees.
Model Context Protocol (MCP)	An MCP server configured for the Codex surface	Direct at the protocol level; partial at the harness level	Server configuration, authentication, approvals, and supported transports can differ.
Subagents	Parallel or delegated agent work, when the selected Codex surface supports it	Partial	Delegation semantics, context sharing, and result synthesis are not necessarily the same.
Permissions	Codex approval and authorization controls	Partial	Approval answers “may this action proceed?” It is not the same as filesystem or network isolation.
Sandboxing	Execution-environment restrictions	Non-equivalent as a product feature	A sandbox limits what a process can do; it does not express the full intent of a permission policy.

	Codex-facing		
Claude Code concept	translation	Status	Main caution
Tool integrations	MCP, built-in tools, connected apps, or a repository-side adapter	Partial	The existence of a connector does not imply that the current session can use it or write through it.

Product names, supported surfaces, configuration locations, approval modes, plugin behavior, and the availability of delegated-agent features change more quickly than repository conventions. Check the current Codex documentation for the exact release and interface you are documenting. This chapter stays concept-first so those checks do not require a rewrite.

7.2 Skills: reusable workflow knowledge

A skill is best understood as a reusable package of instructions, examples, and workflow rules. It is not the same thing as a model, a tool, or a permission grant. A skill can tell an agent how to perform a task; it cannot safely grant access to a production system merely by describing that access.

This is close to a direct equivalent when the Claude Code skill is mostly procedural guidance: When asked to change a database schema:

1. Inspect the migration conventions in this repository.
2. Write a reversible migration.
3. Update the fixture and migration test.
4. Run the database-specific checks.
5. Report the migration and its rollback path.

The translation becomes partial when the Claude Code skill depends on harness-specific behavior. Examples include a special invocation syntax, a particular subagent type, a lifecycle hook, or an assumed tool name. Preserve the workflow and rewrite the parts that refer to those mechanisms. Do not preserve a command name simply because it appears in the old skill.

Keep three kinds of guidance separate:

- **Repository guidance:** conventions that should apply regardless of agent, such as the test command, generated-file policy, or source-of-truth rules. Put this in neutral project documentation or the appropriate instruction files.
- **Workflow guidance:** a reusable procedure for a class of tasks. This is a good candidate for a skill.

- **Harness guidance:** instructions about how a particular agent discovers context, asks for approval, or invokes tools. Keep this close to the harness-specific skill or adapter.

The practical migration pattern is to make the procedure agent-neutral first, then add a short Codex-specific wrapper. The wrapper should explain what the agent must inspect and what it must report, not duplicate the entire repository manual.

Current OpenAI guidance describes a skill as a `SKILL.md` file with optional scripts, references, and assets. Codex skills are available across the CLI, IDE extension, and ChatGPT desktop surfaces, while plugin-bundled skills can also appear in connected ChatGPT or Work experiences. The exact discovery and invocation controls differ, and arbitrary API or SDK workflows are not automatically Codex skills. See the [official skill-building documentation](#) before documenting a directory name, manifest field, or invocation syntax.

7.3 Hooks: automation around the agent

Claude Code hooks are often used for deterministic actions around agent events: checking a command, formatting a file, logging an action, or refusing a risky operation. Codex also has a documented hook framework, but it is not a drop-in Claude Code hook system. Its current documentation describes events such as `PreToolUse`, `PostToolUse`, `PermissionRequest`, `SubagentStart`, `SubagentStop`, and `Stop`, with `PreToolUse` able to block or rewrite a tool call. `PostToolUse` can provide feedback after an action; it cannot undo a command that already ran. See the [Codex hooks documentation](#) for the current configuration and event contract.

Translate hooks by moving each responsibility to the narrowest deterministic layer:

Original hook responsibility	Safer translation target
Format changed source files	Formatter command, pre-commit hook, or CI check
Reject a known-dangerous command	Shell wrapper, restricted runner, container policy, or human approval
Add repository context	Instruction file, skill, or explicit prompt context
Notify a team after a change	CI automation or a separately reviewed integration
Record an audit event	Version-control, command-runner, or organization-level logging

This matters because a model prompt is a poor substitute for a deterministic gate. “Do not run this command” in an instruction file can guide an agent, but it cannot protect against a separate shell, a developer mistake, a compromised dependency, or a different automation path. Put safety-critical controls below the agent.

If the target Codex interface does provide lifecycle hooks, first test a trivial hook that only records an event. Confirm when it runs, what input it receives, whether a non-zero exit blocks the action, and whether it runs in the same sandbox as the command it is observing. Only then move a formatter or policy check into it.

Do not assume that Claude Code hook event names, ordering, input formats, or blocking guarantees carry over to Codex. Test the exact Codex version and surface you use. If a required hook behavior is not supported, use repository scripts, git hooks, CI, or an explicit review step instead.

7.4 MCP: same protocol, different connection

The Model Context Protocol (MCP) is a standard way for an AI application to discover and call tools or retrieve context from an external server. At that protocol level, an MCP server is a direct conceptual equivalent: the same server may be usable from both Claude Code and Codex. Current Codex host documentation describes local STDIO and Streamable HTTP connections, OAuth or bearer authentication, and CLI commands such as `codex mcp add`, `codex mcp list`, and `codex mcp login`. The ChatGPT desktop app, CLI, and IDE share the host configuration model, while hosted plugin tools are a separate surface.

The harness-level mapping is only partial. Each client decides how it stores server configuration, which transports it supports, how it handles authentication, whether it exposes every server capability, and how it asks for approval. A server that works in Claude Code may therefore require configuration or permission changes before Codex can use it.

Treat an MCP connection as three separate questions:

1. **Can the client discover the server?** Configuration and process startup must work.
2. **Can the session call the tool?** The tool may be visible but unavailable because of approval, account, network, or workspace restrictions.
3. **What may the tool do?** A connected tool can have read and write operations, and those operations may have external side effects.

A good first test is read-only and narrow:

Use the configured issue-tracker connection only to read issue ABC-123.

Do not add comments, change fields, transition the issue, or call any other tool.

Report whether the connection is available and quote only the issue title and status.

If that works, test a harmless write separately, with an explicit approval boundary. Do not infer write authority from a successful read. Do not place secrets in an instruction file or prompt when the connector has a supported authentication mechanism.

MCP also does not turn an external system into trusted truth. Validate identifiers, minimize the requested scope, and make the agent report what it actually read or changed. For a production system, prefer an integration with its own audit trail and permission model over a long prompt that says the agent should be careful.

Confirm the current Codex surface’s MCP support, supported transports, configuration method, authentication flow, and whether individual tools require confirmation. These details are volatile. The protocol relationship is stable; the client behavior is not.

7.5 Subagents: delegation is not parallelism by itself

Claude Code users may use subagents to give a bounded task to a separate context: inspect tests, review a design, search for callers, or implement a small independent change. Current Codex releases support subagent workflows in the CLI, IDE extension, and ChatGPT desktop app; the CLI exposes an `/agent` workflow, and delegated workers can inherit or override model and reasoning settings. “Subagent” still does not guarantee the same context isolation, tool access, or result handoff as Claude Code.

Translate a subagent task into an explicit contract:

Role: repository investigator

Scope: inspect only `src/auth/` and its tests

Question: where is token expiry validated, and which tests cover it?

Restrictions: do not edit files, do not run commands that write files

Output: a short report with file paths, line numbers, and missing-test hypotheses

The contract makes the delegation useful even if it runs as a normal second session. It also prevents a common failure mode: asking several workers to modify overlapping files and then trying to merge their incompatible assumptions.

Parallelize only independent work. Good candidates are separate read-only investigations, test discovery, or analysis of disjoint modules. Keep architectural decisions, shared-file edits, migrations, and final integration under one coordinating context unless you have a deliberate merge strategy.

For every delegated task, define:

- the files or subsystem it may inspect;
- whether it may edit anything;
- the evidence it must return;
- how stale or contradictory findings should be reported; and

- who owns the final decision.

The exact Codex support for delegated agents, parallel workers, context sharing, model selection per worker, and result synthesis depends on the interface and current release. Do not promise a particular subagent command or assume that a delegated worker inherits the parent session’s tools and approvals.

7.6 Permissions and sandboxing are different controls

This distinction is the one most likely to cause trouble for a Claude Code user. An approval or permission control is a decision about whether an action is authorized. A sandbox is a restriction on what the process can access or execute even if the agent tries. They overlap in purpose, but they are not substitutes.

Consider a command that deletes temporary files:

- An **approval** asks whether you, the policy, or the harness permits the agent to run it.
- A **sandbox** may prevent it from reaching files outside the working directory, or may restrict network access.
- A **repository rule** may tell the agent not to delete files at all.
- A **backup or version-control policy** may make recovery possible if the other layers fail.

Use all four layers where the consequence justifies them. Do not treat “the agent asked first” as proof that the command is safe, and do not treat “the process is sandboxed” as proof that the requested task is authorized.

Claude Code and Codex can differ in how they express approval modes, allowed commands, writable directories, network access, and escalation. For the current local Codex configuration, the documented sandbox modes are **read-only**, **workspace-write**, and **danger-full-access**; approval policies include **untrusted**, **on-request**, and **never**. The CLI exposes `/permissions`, and `workspace-write` can be configured with writable roots and optional network access. Cloud and ChatGPT web environments have different controls, so do not transfer this local baseline to them without checking their documentation. The visible prompt is not the complete security model.

Before a non-trivial task, state the boundary in plain language:
 You may read and edit files under this repository and run the project's tests.
 Do not access sibling directories, production services, real credentials, or the network.
 Ask before deleting files, changing dependency lockfiles, or making external writes.
 If a test requires unavailable infrastructure, stop and report the missing dependency.

Then verify the boundary with a harmless probe. Check the current working directory,

inspect the intended diff, and confirm that a command requiring network or an external write is actually blocked or prompts for approval. A statement in the prompt is useful policy context, not a replacement for enforcement.

Approval mode names, command-line switches, default sandbox policy, network behavior, writable-directory rules, and escalation behavior are product and release details. Verify them for the exact Codex interface you use. Document the invariant: approval is authorization and sandboxing is execution isolation, even when the implementation changes.

7.7 A migration rule of thumb

When translating a Claude Code setup, move information down this hierarchy only when necessary:

1. Put repository-wide facts and invariants in neutral project documentation.
2. Put repeatable procedures in a skill or script.
3. Put deterministic checks in formatters, tests, git hooks, CI, containers, or policy tooling.
4. Put interactive preferences in the harness-specific instruction file.
5. Put access control and isolation in the approval, operating-system, container, and organization layers.

This produces a smaller and more durable Codex layer. It also lets Claude Code and Codex share the same tests and repository conventions without pretending that their interfaces are identical.

8 Prompting patterns

Claude Code users often need only a short prompt because their project instructions, habits, and preferred interaction mode already supply a lot of context. When they move to Codex, they may either repeat the entire repository manual or give a vague request and expect the agent to infer the same workflow. Both approaches fail for opposite reasons: one buries the task, and the other leaves authority and success criteria ambiguous.

The most portable prompt gives the agent a compact task contract:

Goal: what outcome should exist when the task is complete?

Context: which files, issue, behavior, or constraints matter?

Scope: what may change, and what must remain untouched?

Authority: may the agent edit, run tests, use external tools, or write externally?

Validation: which checks demonstrate success?

Output: what should the final report contain?

Stop conditions: when should the agent pause and ask instead of guessing?

This is not a rigid template. It is a way to expose the decisions that a harness cannot safely

infer. In both tools, a good prompt narrows the search space and defines what “done” means. The Codex version often benefits from stating approval and sandbox boundaries explicitly, especially when the session may run with more autonomy.

8.1 Exploration before editing

The first prompt in a new repository should usually separate orientation from implementation. This gives you evidence about what the agent understood and prevents a plausible but misplaced edit.

Claude Code-style prompt:

Explore the authentication flow and tell me where token expiry is checked.
Don't edit anything yet.

Codex translation:

Inspect the authentication flow, including its tests, and report where token expiry is checked. Do not edit files or run write-producing commands. Include the relevant paths, the current behavior, and any uncertainty. Stop after the report.

The Codex prompt is not “better” because it is longer. It makes the no-edit boundary and evidence requirement explicit. The same wording is useful in Claude Code when the task is high-risk or the repository instructions are unfamiliar.

After reading the report, give the implementation request as a second turn. This creates a clean checkpoint:

Using the findings you just reported, propose the smallest change that makes expired tokens return HTTP 401. Before editing, list the files you expect to change and the tests you will run. Do not implement until I approve the plan.

If your current harness already has a planning mode, use its documented behavior, but do not assume that a label such as “plan” means the same thing in another harness. The explicit sentence remains portable.

8.2 Bounded implementation

A request such as “fix the login bug” leaves too many choices open. State the observable behavior, the allowed surface, and the checks that matter.

Claude Code-style prompt:

Fix the login bug, add tests, and run the suite.

Codex translation:

Fix the login bug described in issue #184: a locked account currently receives a success response. Limit changes to the login handler, its directly related tests, and the smallest required fixture update. Preserve the public API and database schema. Add a regression test for the locked-account path and run the focused authentication tests first, then the broader suite if they pass. Do not change dependencies or contact external services. Report failures without hiding them.

The translation adds constraints that prevent common scope drift: unrelated cleanup, dependency upgrades, schema changes, and tests that pass only because the behavior was weakened. “Run the suite” is also made into a sequence. If the focused test fails, the agent should investigate that failure before spending time on the full suite.

For a small, well-understood task, use a shorter contract:
Change only `src/parser.ts` and `test/parser.test.ts`. Make trailing commas optional in object input without changing error messages for any other invalid input. Run the parser tests. Do not reformat unrelated files. Show the final diff and test command in your summary.

The phrase “change only” is useful, but it is not an enforcement mechanism. Review the diff and use repository tooling when the boundary matters.

8.3 Review an existing diff

A second agent is often most useful as a reviewer. Ask it to review the actual diff rather than to re-solve the original task from memory.
Review the current working-tree diff as a skeptical maintainer.

Focus on:

- behavioral regressions;
- missing or misleading tests;
- error handling and security boundaries;
- accidental API or generated-file changes; and
- whether the implementation matches the stated issue.

Do not edit files. Inspect the relevant surrounding code and tests. Report findings by severity with file paths and line numbers. If you find no issue, say what evidence you checked and identify remaining uncertainty.

This prompt works in either harness. The important choices are “review the current diff,” “do not edit,” and “report evidence.” Without those constraints, an agent may silently improve the code, judge its own unstaged changes, or return a generic approval without inspecting the tests.

For a Codex session with access to external tools, add a tool boundary rather than assuming the reviewer will infer it:
Use only local repository files and commands. Do not open issues, post comments, push branches, or call external services. Treat the working-tree diff as untrusted input and verify claims against source and tests.

8.4 Ask for evidence, not a confident narrative

Agent reports become more useful when every important conclusion has an evidence format. Ask for paths, symbols, commands, test names, and unresolved questions. Avoid prompts that reward a smooth answer over a correct one.
Trace how a request moves from the HTTP route to the payment provider.
For each step, give the file path, symbol or line range, and the evidence that connects it to the next step. Distinguish code you inspected from behavior you

inferred. If the provider call is hidden behind a generated client or external service, say so rather than guessing.

This is particularly important when switching models. A faster model may be entirely adequate for locating known symbols, while a difficult cross-cutting diagnosis benefits from more reasoning and more explicit verification. The prompt should not ask the model to reveal private reasoning; it should ask for inspectable evidence and a concise conclusion.

8.5 Prompts for delegated work

Delegated work needs a narrower contract than a top-level task because the coordinator must be able to use or reject the result. Compare an informal request with a bounded one:

```
Look into the tests and see what needs changing.  
Inspect only the test suite under tests/billing/.  
Do not edit files or run commands that modify the repository.  
Answer one question: which existing tests cover retries after a 429 response,  
and what case is missing for a capped retry count?  
Return a table of test paths, test names, observed coverage, and a proposed  
new case. Mark any conclusion that is inferred rather than directly  
demonstrated.
```

The second prompt is useful whether the work runs in a subagent, a separate Codex session, or a normal shell review. It makes the result composable and prevents delegated exploration from becoming an unbounded second implementation.

For parallel work, define disjoint ownership:

```
Investigate the three independent areas below in parallel. All workers are read-only.
```

```
A: API route and request validation  
B: database model and migration history  
C: integration tests and fixtures
```

```
Return one report per area, then list contradictions without resolving them.  
The coordinator will decide which files to edit. Do not modify shared  
documentation or create generated artifacts.
```

Do not ask several workers to edit the same file unless you have a merge and review plan. Parallel execution reduces elapsed time only when the work is genuinely separable.

8.6 Recovery patterns

The best recovery prompt is specific about the failure and the next evidence to collect. “Try again” throws away useful information and often produces a second version of the same mistake.

8.6.1 The agent changed the wrong files

First inspect the diff. Do not ask the agent to reset the entire working tree if it may contain the user’s changes.

Stop implementation. Inspect the current diff and classify each changed file as:

1. my pre-existing work;
2. your change for this task; or
3. uncertain.

Do not revert anything yet. Explain why each file is in the diff. We will keep pre-existing work and remove only changes that are clearly yours and out of scope.

Once the ownership is clear, give a separate cleanup instruction with the exact files and recovery method. If ownership is uncertain, preserve the file and ask for a manual decision. A broad reset is not a recovery pattern; it is a destructive action.

8.6.2 The tests fail after the change

Make the failure observable before proposing a fix:

The focused tests now fail in `test/parser.test.ts`. Do not edit yet. Re-run only the failing test, capture the exact assertion and stack trace, and compare it with the pre-change behavior visible in the test and implementation. Report whether this is a product regression, an outdated expectation, an environment failure, or an unrelated failure. Then propose one minimal next step.

If the agent already made several speculative changes, ask it to stop and summarize the hypotheses it tried. Do not let it stack another fix on top of an unclassified failure.

8.6.3 The agent claims a tool or capability is unavailable

Separate a missing capability from a missing configuration or permission:

Do not assume the issue tracker is unavailable. Check, in order:

1. whether the configured connection is visible;
2. whether a read-only operation is authorized;
3. whether the requested issue exists; and
4. whether the write operation requires separate approval.

Use no write operation. Report the first failing layer and the exact evidence.

This avoids both unsafe retries and premature conclusions. A tool can be installed but not enabled, visible but read-only, or authorized but unable to reach the requested resource.

8.6.4 The context is stale or contradictory

Ask for a fresh, bounded re-read rather than repeating the entire task:

The previous plan conflicts with the current source: the handler no longer calls `validateSession()`. Re-read the current implementation and the focused tests. Treat the files as authoritative over your earlier summary. State which assumptions are now invalid, then produce a revised plan. Do not edit until the contradiction is resolved.

The phrase “treat the files as authoritative over your earlier summary” is valuable after a long session, a branch switch, a generated-file update, or a hand edit made between turns.

8.6.5 The agent is over-scoping the task

Restate the boundary and require an out-of-scope list:
Return to the original scope: fix the null-handling bug in the CSV importer.
Do not refactor the importer, rename public symbols, update dependencies, or reformat neighboring files. If you believe any of those changes are required, stop and list the dependency with evidence. Continue only with the smallest change that satisfies the regression test.

Do not reward scope expansion merely because the extra cleanup is sensible. Record it as a separate task so that it gets its own review and acceptance criteria.

8.6.6 The agent is too hesitant

Give explicit authority for safe local actions while preserving a stop boundary:
You are authorized to inspect and edit files in this repository and run non-destructive local tests. Make the requested change and validate it. Ask before external writes, destructive commands, dependency changes, or edits outside the repository. If a test cannot run, report the exact blocker and continue with safe static checks where possible.

This is clearer than repeatedly approving routine commands one by one, but it still leaves consequential actions behind an explicit boundary. The exact approval behavior remains harness-specific.

8.7 A compact prompt checklist

Before sending a substantial request, check that it answers the questions that matter for this task:

- What observable outcome should change?
- Which files or subsystem are in scope?
- What must remain unchanged?
- Is the agent allowed to edit, run commands, use external tools, or write externally?
- Which tests or other evidence demonstrate success?
- What should happen if the evidence is unavailable or contradictory?
- What must the final report include?

You do not need all seven lines in every prompt. A one-line typo fix may need only the target, constraint, and test. A multi-file migration needs all of them. The more autonomy, external access, or irreversible state a task involves, the more explicit the contract should be.

The central habit is portable across Claude Code and Codex: ask for inspection before commitment, define authority separately from goals, require evidence, and recover by classifying the failure before making another edit. The interface can change. Those habits remain useful.

Recheck examples that mention a particular Codex mode, command, model control, plugin, skill invocation, hook, MCP behavior, or delegated-agent workflow before publication. Keep the prompts themselves generic enough to survive product changes, and link to current official documentation where a reader needs exact syntax.

9 Migration cookbook

The first migration mistake is treating a second coding agent as a second copy of the repository’s operating manual. That creates two instruction files that slowly diverge, two interpretations of the same safety rules, and a debugging problem when Claude Code and Codex appear to behave differently. A safer migration starts with a small adapter, establishes where shared guidance lives, and expands only after a low-risk task has gone through the complete review loop.

This cookbook assumes that Claude Code remains the primary harness. Codex becomes another way to inspect, change, test, and review the same repository. The aim is coexistence, not a wholesale rewrite of the project’s tooling.

9.1 Add the smallest useful AGENTS.md

If a repository already has a carefully maintained `CLAUDE.md`, begin with a bridge. Put it at the repository root so Codex can discover it when it starts in the project, and make its purpose explicit:

```
# Repository instructions
```

The canonical project instructions are in `[`CLAUDE.md`](CLAUDE.md)`. Read and follow that file for repository work.

This file is intentionally a thin compatibility layer for agents that discover ``AGENTS.md``, so the project can support Codex without maintaining a second copy of its instructions.

This bridge is useful because it makes the ownership decision visible. It tells a human reviewer that `AGENTS.md` is not a competing policy document, and it instructs the agent to read the established instructions. Keep the bridge short enough that somebody can understand its role in one glance.

Do not copy a large `CLAUDE.md` into `AGENTS.md`. Duplication looks convenient during the first migration and becomes a maintenance defect as soon as a command, directory name, test rule, or safety constraint changes in one file but not the other. A duplicated instruction can also be more dangerous than a missing instruction: the agent receives two plausible rules and has to guess which one wins.

The bridge is an instruction, not a guarantee that every harness will implement file references in exactly the same way. Test it with a harmless inspection request and ask the agent to identify the repository’s build command, source-of-truth document, and a constraint that appears in `CLAUDE.md`. If the answer shows that the bridge is not being followed reliably, use a shared neutral document for the important rules rather than adding more prose to the adapter.

The bridge remains a small instruction that asks the agent to read another file; it is not a documented file-include primitive. Test it with a harmless inspection request. If the agent does not reliably follow the reference, put the important shared rules in a neutral document or reproduce a small, maintained subset in `AGENTS.md`.

9.2 Choose a shared-instruction strategy

There are three reasonable arrangements. The bridge pattern is the least disruptive when Claude Code is the established primary harness. A shared-neutral pattern is stronger when both harnesses will make frequent changes. A split pattern is appropriate only for genuinely harness-specific behavior.

In the bridge pattern, `CLAUDE.md` remains canonical and `AGENTS.md` points to it. Repository rules, build commands, style guidance, and safety constraints stay where they are. This is a good first step for an existing project because it changes discovery without reorganizing the project.

In the shared-neutral pattern, stable repository rules move into a neutral file such as `docs/agent-instructions.md`. Each harness-specific file becomes a short adapter that names the shared file and adds only the behavior specific to that harness. The neutral file should contain facts that remain true regardless of who is acting: which directories are source, how to run tests, what must not be modified, and how generated artifacts are handled.

In the split pattern, the shared file still owns repository policy, but `CLAUDE.md` and `AGENTS.md` contain separate instructions for different interfaces. Examples include a Claude Code hook convention, a Codex approval workflow, or a command wrapper available in only one environment. Split only the interface detail. Do not split the underlying safety rule or definition of done.

A practical repository layout looks like this:

```
repository/  
  CLAUDE.md           # primary Claude Code adapter or instructions  
  AGENTS.md           # Codex adapter  
  docs/
```

```

    agent-instructions.md      # optional neutral shared guidance
.claude/                      # Claude-specific skills, hooks, and settings
.codex/                       # Codex-specific settings, if used
Justfile                      # shared build entry point

```

The layout is illustrative, not a requirement. Do not create empty `.claude/` or `.codex/` directories merely to make the tree look symmetrical. Add a directory when it contains configuration or reusable behavior that the corresponding harness actually needs.

For this primer’s repository, the existing project rules are the important shared context: the primer layout, containerized rendering requirement, `STYLE_GUIDE.md`, and the instruction to preserve unrelated work. A Codex bridge should point at those rules rather than restating them. If a rule later proves useful to both harnesses but is currently buried in Claude-specific prose, extract that rule into a neutral document and make both adapters reference it.

9.3 Roll out in a branch, one capability at a time

Do the first Codex run on a branch or worktree with a clean, known starting point. The goal of the first run is to learn how the harness behaves in this repository, not to complete the largest task on the backlog. A documentation typo, a focused test addition, or a small refactor with a clear test command gives you enough signal without creating a large review surface.

Start with inspection. Ask Codex to read the instruction bridge, inspect the repository status, identify the relevant files, and propose a plan without editing anything. Compare its description with the project’s actual conventions. This catches instruction-discovery problems before they become code changes.

Then give it a bounded implementation task. State the allowed area, the required verification command, and the stop condition. A useful first prompt has this shape:
Inspect the repository instructions and current git status first.

```

Task: update the primer index so it links to the existing HTML document.
Scope: README.md and the one project file needed for the link. Do not change
generated outputs, unrelated primers, or instruction files.
Verification: run the relevant link or formatting check, if one exists, and
show the final diff and test result. Stop before committing.

```

Review the diff as if a human had submitted it. Check that the agent respected scope, used the repository’s commands, and reported failures honestly. Only after that should you try a task that edits source code, runs a longer build, or uses network access.

A staged rollout can be summarized as a sequence:

- Discover: inspect instructions, status, structure, and available checks.
- Propose: identify files, risks, and a verification plan.

- Change: make the smallest coherent edit.
- Verify: run focused checks before broad checks.
- Review: inspect the diff, generated files, and command output.
- Expand: permit a larger task only when the earlier stage behaved predictably.

Treat each stage as evidence, not ceremony. If Codex edits files before the inspection request is complete, ignores a scope boundary, or claims a check passed without showing a result, stop and fix the workflow before increasing the task size.

9.4 Port integrations by behavior, not by filename

Claude Code skills, hooks, Model Context Protocol (MCP) servers, subagents, and permission settings may have analogues in Codex, but an analogous concept is not necessarily a drop-in replacement. Port the behavior you need and test the smallest useful version of it.

For a skill, first write down the input, output, files it may touch, commands it may run, and failure behavior. Then decide whether Codex has a supported mechanism for that workflow. If it does, port the narrow workflow and preserve the original Claude Code version until the new one has passed the same acceptance checks. If it does not, keep the Claude-specific skill and expose the underlying command or document as a normal repository workflow.

For a hook, distinguish enforcement from convenience. A hook that prevents commits containing generated intermediates is an enforcement mechanism; a hook that prints a reminder is convenience. Preserve enforcement in a tool-neutral place such as a git hook, CI check, or build recipe when possible. A harness-specific reminder can remain harness-specific.

For MCP, inventory the data and actions the server provides before changing configuration. A read-only documentation server and a deployment server have very different risk profiles. Port the read-only case first, confirm authentication and network behavior, and do not grant write capabilities merely because the old harness had them.

For subagents or parallel workers, begin with independent read-only tasks. Parallelize searches, test inspection, or candidate design notes before parallelizing edits. If two workers can modify the same file, their work needs explicit ownership and a merge strategy; otherwise parallelism creates conflicts that cost more time than it saves.

Keep a migration note for each port. It should say what behavior was required, where it now lives, what remains Claude-specific, and which test demonstrates that the behavior works. That record prevents a future maintainer from assuming feature parity based on similar names.

9.5 Preserve a clean rollback path

The migration should be removable without reconstructing the repository. Keep the initial bridge as a small, reviewable commit. Keep configuration changes separate from source changes. Avoid changing the primary build system and adding Codex configuration in the same commit unless the task requires both.

If a Codex run produces an unexpected result, stop the run, save the transcript or command log if your local workflow permits it, and inspect the diff. Revert only the files created by that run; do not discard pre-existing user changes. If the problem came from a shared instruction, fix the instruction and repeat the smallest reproducer. If it came from a harness-specific setting, disable that setting before trying a broader task.

The useful rollback unit is not “remove Codex.” It is “remove this adapter, integration, or permission change.” That keeps the primary Claude Code workflow intact while you learn.

10 Safety, cost, and operational discipline

Adding another agent changes the number of paths by which commands, credentials, generated files, and external services can be reached. The risk does not come only from a model making a bad edit. It also comes from unclear authority: the agent may be allowed to run a command, but the repository may not make clear whether that command is appropriate; the sandbox may permit a file write, but the task may not authorize it.

10.1 Separate approval from sandboxing

Approval answers whether a proposed action may proceed. Sandboxing answers what the execution environment can reach or modify. They are related controls, but they are not interchangeable. An approved command can still fail because the sandbox blocks it. A command inside a permissive sandbox can still be inappropriate because nobody reviewed its scope.

For example, a containerized PDF build may be an approved repository operation, but it can still be unsafe if the container receives unnecessary credentials or mounts a broad directory. Conversely, a read-only search may be safe to permit automatically even when a later write or network request requires review.

Make the authority boundary visible in the task prompt and in the repository instructions. Say which paths may change, whether generated files are expected, whether network access is needed, and whether the agent may commit. “Fix the build” is not an operational boundary.

“Inspect the failing primer, change only its source and Justfile, run the containerized build, and stop before committing” is.

10.2 Protect secrets and external systems

Do not assume that a second harness inherits the first harness’s secret-handling behavior. Audit environment variables, credential helpers, configuration files, MCP connections, cloud CLIs, container mounts, and shell startup files. A repository may be safe for local edits but unsafe for deployment commands if the agent can reach production credentials.

Use least privilege for integrations. Give an agent read-only access to issue tracking before write access, a staging credential before a production credential, and a narrowly scoped service account before a personal account. If a task does not need an external service, do not connect it for convenience.

Treat pasted logs and generated files as possible secret carriers. A failed command can expose a token in its output, and a generated report can embed environment details. Review artifacts before sharing them with another harness or committing them.

10.3 Control cost with task boundaries

Cost is not only the provider’s per-token charge. It includes repeated exploration, long-running commands, review time, failed builds, duplicated work between agents, and the opportunity cost of an engineer supervising an unbounded loop. A fast model that needs three correction cycles may be more expensive than a slower model that completes a bounded task correctly.

Give each task a stopping rule. Ask for an investigation report before authorizing a broad edit. Set a maximum useful iteration count for debugging. Split a large migration into independently reviewable changes. Stop when the acceptance criteria are met rather than asking an agent to “keep improving” without a definition of done.

Choose model and reasoning settings by task risk and ambiguity. Use a quicker setting for a mechanical rename with strong tests; reserve deeper reasoning for a poorly understood failure, a cross-cutting design decision, or a review where missed edge cases matter. Exact model names, availability, limits, pricing, and plan entitlements belong in a separately verified table, not in timeless repository instructions.

Verify before publication (volatile): Verify current Codex model identifiers, model-selection commands, reasoning controls, usage limits, pricing, plan availability, and API-versus-subscription differences. Do not preserve an unverified

number or command in the final primer merely because it was correct during drafting.

10.4 Make generated changes observable

Generated HTML, PDF, lockfiles, snapshots, database migrations, and vendored assets can make a small source edit look large. Before starting, identify which outputs are expected and how to regenerate them. Afterward, compare both source and generated changes.

In a primer repository, the source Markdown and thin `Justfile` are inputs, while HTML and PDF are committed outputs. A safe task says which of those files may change and which build recipe produces them. The agent should not silently replace an output with a host-generated artifact when the repository requires containerized rendering.

Use `git diff --stat`, the normal diff, and the repository's verification commands. A non-trivial output file is not proof of a correct render; a successful command is not proof that the output contains the intended content.

10.5 Establish an operating rhythm

The safest two-harness workflow has a predictable rhythm: inspect, plan, implement, test, review, and record. The record can be lightweight, but it should capture enough information to explain why a particular agent, model, or permission setting was used.

For recurring work, keep a small task ledger with the date, repository revision, harness, model setting, task class, result, review burden, and notable failure. This turns anecdotes into evidence and makes regressions visible when a harness, model, instruction file, or dependency changes.

Before a release or a destructive operation, use a human checkpoint even if the agent has passed every automated check. The checkpoint is where you confirm the target environment, migration order, data impact, and rollback procedure. Automated verification can establish that a command works; it cannot establish that today is the right day to run it against production.

10.5.1 Safety and operational checklist

- Start from a known branch, worktree, or commit.
- Read the instruction bridge and confirm the source of truth.
- State the allowed files, forbidden files, and expected generated outputs.
- Separate approval decisions from sandbox or environment limits.

- Remove unnecessary secrets, mounts, credentials, and network access.
- Start with read-only inspection and a low-risk change.
- Give long-running tasks a stopping rule and a verification command.
- Review the complete diff, including generated artifacts and deletions.
- Do not commit or deploy without an explicit human checkpoint.
- Record the harness, model setting, repository revision, result, and failures.
- Keep configuration and source changes in separate, recoverable commits.
- Verify volatile product claims before publication or team rollout.

11 What to measure

“It felt better” is a useful first impression and a poor evaluation method. Claude Code and Codex can produce different experiences because of the harness, the underlying model, the instruction files, the permissions, the task prompt, or simple task-order effects. A fair bake-off controls those variables well enough to answer a narrower question: which setup is more reliable for this repository and this class of work?

11.1 Define the comparison before running it

Write the evaluation question first. “Is Codex better?” is too broad to measure. “For containerized primer maintenance, does Codex complete focused source edits with equal correctness and lower review burden?” is testable. “Which harness should handle unfamiliar-repository exploration?” is another useful question, but it needs a different task matrix.

Use the same repository revision, task brief, acceptance criteria, test commands, and starting information for each run. Give each setup a fresh session unless session continuity is itself the subject of the experiment. Record the model and reasoning setting separately from the harness so a model change is not mistaken for a harness effect.

Randomize the order when practical. If Claude Code always runs first, it may benefit from a clean task and Codex may benefit from clues left in the working tree, or vice versa. The simplest control is to use separate worktrees created from the same commit and alternate which harness runs first across tasks.

Do not let one agent repair the other agent’s working tree before scoring. That measures a collaborative workflow, not independent performance. Collaborative runs are valuable, but label them as a separate experiment.

11.2 Use a task matrix that resembles real work

A useful matrix contains tasks with different failure modes, not ten versions of the same rename. Include a focused bug fix with an existing regression test, a small feature with acceptance tests, a cross-file refactor, an unfamiliar subsystem investigation, a documentation or primer edit, a build failure, a review-only task, and a task that must correctly refuse an unsafe or out-of-scope action.

For this repository, representative tasks could include adding a primer while preserving the standard layout, correcting a Markdown-to-PDF rendering issue, updating a `Justfile` without installing host dependencies, reviewing a generated-output diff, or identifying why a section reference became stale after heading changes. Each task should have a known verification path and a clear boundary around committed outputs.

The task brief should contain what a real engineer would know at the start, but not the solution. Include the issue description, relevant user-facing behavior, repository constraints, and definition of done. Do not give one harness extra hints because its interface makes you more comfortable.

11.3 Score correctness before speed

A fast wrong answer is not a successful coding task. Score correctness and safety first, then use time and cost to distinguish among successful runs. A simple four-point rubric works well:

Dimension	0	1	2	3	4
Functional correctness	Does not address the task	Major failure	Partially works	Works with a minor issue	Meets acceptance criteria and edge cases
Scope discipline	Unusable or destructive scope violation	Significant unrelated changes	Some unnecessary changes	Mostly bounded	Only necessary files and behavior changed

Dimension	0	1	2	3	4
Verification	No useful verification or false claim	Verification mostly missing	Partial or weak checks	Appropriate checks run	Checks run, results reported, and failures handled honestly
Maintainability	Makes future work harder	Fragile or opaque result	Acceptable with cleanup needed	Clear and conventional	Fits project conventions and improves clarity
Safety behavior	Unsafe action or secret exposure	Ignores a material warning	Needs intervention	Respects stated boundaries	Correctly asks, refuses, or narrows risky work

Record the raw scores and a short justification. Do not hide a safety failure inside an average: a run that exposes credentials or modifies an unauthorized target should fail the safety gate regardless of its other scores.

11.4 Record the review burden

Review burden measures the human work required after the agent stops. Count the number of correction cycles, files that needed manual cleanup, tests that had to be added by the reviewer, and minutes spent understanding the diff. A small diff can still have a high review burden if the explanation is unclear or the agent changed a subtle invariant.

Record time in separate phases where possible: time to first useful plan, time to implementation, command or build time, and human review time. Record provider-reported usage or cost only when it is available and label its source. UI estimates, API billing, and local wall-clock time are different measurements.

Verify before publication (volatile): Confirm how current Codex and Claude Code surfaces expose usage, token counts, rate limits, model metadata, and cost. Product interfaces and billing terminology may change, so the evaluation template should leave room for “not available” rather than inventing precision.

11.5 Report distributions, not a single victory lap

Run enough tasks to expose variation. One impressive result can be an outlier, and one failure can be caused by a transient service or dependency problem. Report per-task outcomes and aggregate summaries such as median completion time, median review time, success rate, scope-violation count, and safety-gate failures.

Separate task classes. A harness that is excellent at repository exploration may not be the best choice for a long PDF build, and a model that handles ambiguous design work may be wasteful for a mechanical edit. The useful conclusion may be a routing rule rather than a winner:

If the task is a bounded edit with strong tests, use the faster setup.

If the task is ambiguous or cross-cutting, use the setup with the lower review burden.

If the task touches credentials, deployment, or destructive data operations, require the stricter approval path and a human checkpoint regardless of benchmark score.

State uncertainty. A five-task result is a pilot, not a general law. Mention missing data, infrastructure failures, manual interventions, and any task where the comparison was not equivalent.

11.6 A practical evaluation record

Keep one record per run. Plain text or a spreadsheet is enough; the important property is consistent fields.

```
date: 2026-08-21
repository_revision: "<commit or worktree identifier>"
task_id: primer-render-fix-01
harness: Codex
model: "<record exact identifier at run time>"
reasoning_setting: "<record exact setting, if exposed>"
permissions: "<summary>"
network: "disabled"
result: pass
scores:
  correctness: 4
  scope: 4
  verification: 4
  maintainability: 3
  safety: 4
wall_clock_minutes: 18
review_minutes: 7
correction_cycles: 1
tests: "<commands and outcomes>"
notes: "Changed source and regenerated outputs in the required container."
```

The date and identifiers in this example are placeholders for a real run record. Do not treat the model, setting, or result values as a claim about product performance.

11.6.1 Fair bake-off checklist

- Define one repository-specific evaluation question.
- Select a mixed task matrix with realistic failure modes.
- Pin the same repository revision and starting information.
- Use separate worktrees or restore the exact starting state.
- Record harness, model, reasoning setting, permissions, and network state.
- Randomize task order when practical.
- Keep independent runs independent; score collaboration separately.
- Score correctness and safety before speed or cost.
- Record review time, correction cycles, and manual cleanup.
- Report per-task results and distributions, not only an average.
- Separate transient infrastructure failures from agent failures.
- Mark conclusions as pilot evidence when the sample is small.

12 Conclusion and quick reference

Codex does not need to replace Claude Code to be useful. The practical opportunity is to add a second harness without creating a second, conflicting repository culture. Keep the primary instructions authoritative, use a thin `AGENTS.md` bridge or a shared neutral document, and make harness-specific configuration earn its place through a real workflow.

The migration becomes manageable when you treat it as an operational change rather than a file rename. Start with inspection, give the agent bounded authority, verify generated and source changes, and preserve a rollback path. When you compare the tools, compare complete setups on representative tasks. The model, harness, prompt, instruction files, permissions, and review loop all contribute to the result.

The end state is not “Claude Code versus Codex.” It is a routing decision based on the task. One harness may be more convenient for a familiar workflow, another may be better for a particular model or reasoning setting, and a human checkpoint remains necessary wherever the consequences exceed the review surface. The repository should make those choices explicit enough that another engineer can reproduce them.

12.1 Quick-reference: adding Codex to an existing repository

12.1.1 Minimal migration

- Keep `CLAUDE.md` as the canonical instruction source if it already works.
- Add a short root `AGENTS.md` that points to `CLAUDE.md`.

- Ask Codex to inspect instructions and status without editing.
- Run one low-risk, bounded task in a branch or separate worktree.
- Review the diff and verification output.
- Add shared-neutral guidance only when a rule genuinely belongs to both harnesses.
- Port skills, hooks, MCP, or subagents one behavior at a time.
- Record what worked, what remained harness-specific, and how to roll back.

12.1.2 Task-start prompt

Read the repository instruction files and inspect git status before editing.

Task: <one bounded outcome>

Allowed scope: <files or directories>

Do not change: <generated files, unrelated projects, or other exclusions>

Definition of done: <observable acceptance criteria>

Verification: <exact test, build, or inspection command>

Before finishing: show the diff, report verification results, and stop before committing unless I explicitly ask you to commit.

12.1.3 Preflight checklist

- What repository revision am I starting from?
- Which instruction file is canonical?
- Which files may change?
- Which files must not change?
- Does the task need network access, credentials, containers, or external tools?
- What command proves the task is complete?
- What is the rollback point?
- What action requires a human checkpoint?

12.1.4 Postflight checklist

- Is the diff limited to the intended scope?
- Did the agent change source, generated output, or both as expected?
- Did the prescribed check actually run?
- Do the results support the claim of completion?
- Did any command expose secrets or alter external state?
- Are new files, deletions, permissions, and symlinks intentional?
- Is the result reviewable by somebody who did not watch the session?
- Should this run be recorded in the evaluation ledger?

12.1.5 Choosing a harness or model setting

- Use the setup with the lowest review burden for ambiguous or cross-cutting work.

- Use a faster setting for bounded mechanical work with strong automated checks.
- Prefer the stricter approval path for credentials, deployment, destructive commands, and external writes.
- Keep the model identifier and reasoning setting in the task record.
- Do not infer a general winner from one task or from a different repository.
- If the task is unsafe or underspecified, clarify or narrow it before choosing a model.

12.1.6 Volatile-claim verification list

Before publishing this primer, verify from current official documentation:

- How Codex discovers, layers, and prioritizes `AGENTS.md` and related instructions.
- Whether an `AGENTS.md` reference to `CLAUDE.md` is followed as expected in the supported surfaces.
- Current Codex interfaces, installation and authentication steps, and command names.
- Current model identifiers, availability, reasoning controls, and context limits.
- Current plans, pricing, quotas, rate limits, and API-versus-subscription behavior.
- Current support and configuration semantics for skills, hooks, MCP, subagents, approvals, and sandboxes.
- Current usage and billing telemetry available for bake-off records.

Until those checks are complete, keep the primer’s advice architectural and repository-oriented. A bridge, a bounded rollout, an explicit safety boundary, and a fair evaluation method remain useful even when product names or controls change.

13 References

- [Codex CLI documentation](#)
- [Codex model and reasoning guidance](#)
- [Codex project instructions with `AGENTS.md`](#)
- [Codex configuration reference](#)
- [Codex skills](#)
- [Codex hooks](#)
- [Codex MCP integrations](#)
- [Codex subagents](#)
- [Codex sandboxing](#)
- [Codex cloud environments](#)
- [Claude Code documentation](#)